Guia MVC para PokeRush

Estructura de carpetas y criterios para programar sin implementar funcionalidad inicial

Proyecto	Grupo4_PokemonBossRush_ESTFA_2026
Motor	Unity
Lenguaje	C#
Ubicacion	Juego/PokeRush/Assets/Scripts
Fecha	26/05/2026

Alcance. Esta guia documenta solamente la estructura MVC. No define clases concretas ni reglas de combate; esas decisiones se agregaran cuando el equipo empiece a programar funcionalidades reales.

1. Objetivo de la estructura MVC

La estructura MVC separa responsabilidades para que el proyecto crezca de forma ordenada. La idea es que las reglas del juego, la presentacion visual y la coordinacion de acciones no queden mezcladas en una misma carpeta o clase.

Capa	Responsabilidad	No deberia hacer
Model	Guardar estado y reglas del dominio.	Depender de escenas, UI, Input o GameObject.
View	Mostrar informacion y feedback visual.	Calcular reglas del juego o decidir resultados.
Controller	Coordinar input, modelos y vistas.	Guardar datos permanentes o concentrar toda la logica.
Data	Contener configuracion editable del juego.	Ejecutar reglas complejas en tiempo de juego.
Service	Resolver tareas reutilizables.	Representar una pantalla o entidad especifica.
Core	Compartir tipos comunes entre capas.	Convertirse en una carpeta general para todo.

Regla simple. Si el codigo puede probarse sin abrir una escena de Unity, normalmente va en Models. Si necesita objetos de Unity, UI o animaciones, probablemente va en Views o Controllers.

2. Carpetas creadas

Carpeta	Uso esperado
Assets/Scripts/Core	Tipos compartidos, constantes o contratos usados por varias capas.
Assets/Scripts/Models	Estado y reglas puras del juego.
Assets/Scripts/Views	Componentes visuales de Unity: UI, sprites, animaciones y feedback.
Assets/Scripts/Controllers	Componentes que reciben acciones y coordinan modelos con vistas.
Assets/Scripts/Data	Datos configurables desde Unity, como futuros ScriptableObject.
Assets/Scripts/Services	Servicios reutilizables, como guardado, carga de escenas o audio global.

3. Como programar en base a esta estructura

1. Antes de crear una clase, definir que responsabilidad tiene: regla, visual, coordinacion, datos o servicio.
2. Crear la clase en la carpeta correspondiente, usando un nombre que indique su rol.
3. Mantener los modelos independientes de Unity siempre que sea posible.
4. Hacer que las vistas reciban datos para mostrarlos, sin decidir reglas del juego.
5. Usar controladores para conectar acciones del jugador, cambios del modelo y actualizaciones de vista.
6. Mover datos editables a Data cuando deban modificarse desde el Inspector.

4. Criterios para ubicar codigo

Pregunta	Carpeta sugerida
Es una regla del juego o estado interno?	Models
Tiene referencias a UI, Animator, SpriteRenderer o GameObject?	Views o Controllers
Recibe input o coordina una accion?	Controllers
Es informacion editable por disenadores desde Unity?	Data
Sirve a varias partes del juego, como guardar o cargar?	Services
Es un tipo compartido por varias capas?	Core

5. Convenciones recomendadas

Tipo	Convencion de nombre
Modelo	Nombre + Model
Vista	Nombre + View
Controlador	Nombre + Controller
Datos configurables	Nombre + Data o Nombre + Definition
Servicio	Nombre + Service

Importante. Estas convenciones son una guia, no funcionalidad. Todavia no se agregan scripts de juego; solo queda lista la ubicacion para que el equipo programe de forma ordenada.

6. Checklist antes de agregar una funcionalidad

Revision	Resultado esperado
Se entiende que capa necesita el cambio?	Si, antes de crear archivos.
La regla no queda mezclada con UI?	Si, va separada de la vista.
La vista no decide resultados del juego?	Si, solo muestra estado.
El controlador no concentra todo?	Si, coordina sin reemplazar al modelo.
Los datos configurables estan separados?	Si, van a Data cuando corresponda.

7. Resumen

Models guarda reglas y estado. **Views** muestra informacion. **Controllers** coordina acciones. **Data** contiene configuracion. **Services** ofrece utilidades reutilizables. **Core** comparte tipos comunes. Esta version deja solo la estructura, sin implementar mecanicas de juego.